Project Scheduling

The project consists of a number of tasks which needs to be scheduled. However, some tasks can only be initiated when others have been completed. We use an **incidence** matrix to keep track of these relations. This is a matrix with boolean values only 0s and 1s. There will be as many columns and rows as there are tasks and an entry in the matrix then contains the information about precedence restrictions for the task represented by the column and the task represented by the row. More specific details can be found below when presenting $Predecessors$.

Problem

- Minimize the completion time.

Sets

- $t \in Tasks = \{"Remove\ Stuff", "Shet\ Demolition", "Pipe\ Work", "Electric\ Work", "Flag\ Stones", "Shet\ Foundations", "Wood\ Frames", "Painting", "Roofing", "Roofing\ Felt", "Interior\ Installations", "Insert\ Stuff", "Hand\ Over"\}$

Parameters

- Incidence matrix $Predecessors$: Entry $Predecessors_{t2,t1} = 1$ if task $t2$ should be preceded by task $t1$ and 0 otherwise.
- $Duration_t$: The number of days it takes to do task t

Decision variables

- The starting day of task t: $x_t \geq 0$.

Model

Objective:

- Minimize the starting time of the last task T ("$Hand\ Over$"):

$$x_T$$

Constraints:

- Precedence constraint:

$$x_{t1} + Duration_{t1} \leq x_{t2}\ \forall t1, t2 | Predecessors_{t2,t1} = 1$$

Notice the condition in the constraint: Only for tasks where $t1$ has to be completed before $t2$ does the constraint exist.

The full model in Julia/JuMP, available with the name

`ProjectScheduling1.jl`

from the book web-site, is given below:

```
#*************************************************************************
# Project Scheduling Assignment,
#"Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
#*************************************************************************

#*************************************************************************
# PARAMETERS

# Tasks to be performed for the shet building
Tasks = ["Remove Stuff","Shet Demolition","Pipe Work","Electric Work",
        "Flag Stones","Shet Foundations","Wood Frames","Painting","Roofing",
        "Roofing Felt", "Interior Installations","Insert Stuff","Hand Over"]
T=length(Tasks)

# duration time in days to do the job
Duration = [1,3,5,3,7,4,3,1,1,1,3,1,0]

Predecessors=zeros(T,T)

#Tasks that must be done before task1
```

```
#Tasks that must be done before task2
Predecessors[2,1]=1

#Tasks that must be done before task3
Predecessors[3,2]=1

#Tasks that must be done before task4
Predecessors[4,2]=1

#Tasks that must be done before task5
Predecessors[5,3]=1
Predecessors[5,4]=1

#Tasks that must be done before task6
Predecessors[6,3]=1
Predecessors[6,4]=1

#Tasks that must be done before task7
Predecessors[7,6]=1

#Tasks that must be done before task8
Predecessors[8,7]=1

#Tasks that must be done before task9
Predecessors[9,7]=1

#Tasks that must be done before task10
Predecessors[10,9]=1

#Tasks that must be done before task11
Predecessors[11,9]=1

#Tasks that must be done before task12
Predecessors[12,9]=1

#Tasks that must be done before task13
Predecessors[13,5]=1
Predecessors[13,8]=1
Predecessors[13,10]=1
Predecessors[13,11]=1
Predecessors[13,12]=1
#*****************************************************************************

#*****************************************************************************
# Model
```

```
schedule = Model(HiGHS.Optimizer)

@variable(schedule, 0 <= x[t=1:T]) # start time of task t

# Minimize storage cost
@objective(schedule, Min, x[T])

# force predecessor relation
@constraint(schedule, [t1=1:T,t2=1:T; Predecessors[t2,t1]==1],
            x[t1] + Duration[t1] <= x[t2]
            )

print(schedule)
#*****************************************************************************

#*****************************************************************************
# solve
optimize!(schedule)
println("Termination status: $(termination_status(schedule))")
#*****************************************************************************

#*****************************************************************************
# Report results
println("---------------------------------------");
if termination_status(schedule) == MOI.OPTIMAL
    println("RESULTS:")
    println("Objective: $(objective_value(schedule))")
    for t=1:T
        println("Task: \t", Tasks[t], " starting-day: \t", value(x[t]), "\tDuration: \t", Du
    end
else
  println("  No solution")
end
println("---------------------------------------");
#*****************************************************************************
```

Comments to the Julia/JuMP program:

- Notice: The use of conditional constraint construction in the predecessor constraint.

In the second part of the assignment, the speed-up by paying extra is investigated. Notice, there is a budget of how much can be spend on the speedup. Here we only give the additions and changes to the model. The objective stays the same.

Problem

- Minimize the completion time, given a budget for speedup.
- $MaxSpeedUp_t$: The maximal number of days that task t can be sped up.
- $OverTimePay_t$: The extra cost for each day which is sped up.
- $Budget$: How much money we are willing to pay for faster completion. This can either be 5000, 10000 or ∞ (no budget)

Decision variables

- How many days to speed up task t: $0 \leq y_t \leq MaxSpeedUp_t$.
  Notice here we set the upper bound in the variable definition. This could also have been done using an additional constraint.

Constraints:

- Precedence constraint including speed up of tasks:

$$x_{t1} + Duration_{t1} - y_{t1} \leq x_{t2} \ \forall t1, t2 | Predecessors_{t2,t1} = 1$$

- Budget constraints for the overtime pay:

$$\sum_t OverTimePay_t \cdot y_t \leq Budget$$

The full model in Julia/JuMP, available with the name

`ProjectScheduling2.jl`

from the book web-site, is given below:

```
#**************************************************************************
# Project Scheduling Assignment,
# "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
#**************************************************************************

#**************************************************************************
# PARAMETERS

# Tasks to be performed for the shet building
Tasks = ["Remove Stuff","Shet Demolition","Pipe Work","Electric Work",
```

```
        "Flag Stones","Shet Foundations","Wood Frames","Painting","Roofing",
        "Roofing Felt", "Interior Installations","Insert Stuff","Hand Over"]
T=length(Tasks)

# duration time in days to do the job
Duration = [1,3,5,3,7,4,3,1,1,1,3,1,0]

MaxSpeedUp=[0,1,3,1,4,2,2,0,0,0,2,0,0]

Budget=10000

OverTimePay=[0,500,2000,4000,1000,1000,1500,0,0,0,1500,0,0]

Predecessors=zeros(T,T)

#Tasks that must be done before task1

#Tasks that must be done before task2
Predecessors[2,1]=1

#Tasks that must be done before task3
Predecessors[3,2]=1

#Tasks that must be done before task4
Predecessors[4,2]=1

#Tasks that must be done before task5
Predecessors[5,3]=1
Predecessors[5,4]=1

#Tasks that must be done before task6
Predecessors[6,3]=1
Predecessors[6,4]=1

#Tasks that must be done before task7
Predecessors[7,6]=1

#Tasks that must be done before task8
Predecessors[8,7]=1

#Tasks that must be done before task9
Predecessors[9,7]=1

#Tasks that must be done before task10
Predecessors[10,9]=1
```

```
#Tasks that must be done before task11
Predecessors[11,9]=1

#Tasks that must be done before task12
Predecessors[12,9]=1

#Tasks that must be done before task13
Predecessors[13,5]=1
Predecessors[13,8]=1
Predecessors[13,10]=1
Predecessors[13,11]=1
Predecessors[13,12]=1

Budget=5000
#Budget=10000
#Budget=5000000
#*************************************************************************

#*************************************************************************
# Model
schedule = Model(HiGHS.Optimizer)

@variable(schedule, 0 <= x[t=1:T]) # start time of task t

@variable(schedule, 0 <= y[t=1:T] <= MaxSpeedUp[t] ) # save time

# Minimize storage cost
@objective(schedule, Min, x[T])

# force predecessor relation
@constraint(schedule, [t1=1:T,t2=1:T; Predecessors[t2,t1]==1],
            x[t1] + Duration[t1] - y[t1] <= x[t2]
            )

@constraint(schedule,
            sum( OverTimePay[t]*y[t] for t=1:T) <= Budget
            )

print(schedule)
#*************************************************************************


#*************************************************************************
# solve
optimize!(schedule)
println("Termination status: $(termination_status(schedule))")
```

```
#*****************************************************************************

#*****************************************************************************
# Report results
println("--------------------------------------");
if termination_status(schedule) == MOI.OPTIMAL
    println("RESULTS:")
    println("Objective: $(objective_value(schedule))")
    #println("sol x: ", value.(x))
    #println("sol y: ", value.(y))
    for t=1:T
        println("Task: \t", Tasks[t], " starting-day: \t",
        round(value(x[t]),digits=1), "\tDuration: \t $(Duration[t]-value(y[t]))
        $(value(y[t])>0 ? "(reduced by $(value(y[t])))" : "") ")
    end
else
  println("  No solution")
end
println("--------------------------------------");
#*****************************************************************************
```

Comments to the Julia/JuMP program:

- Notice: You have to run the model for each different value of the *Budget*. When having no budget you can either set the budget higher than overtime could ever cost (as done here) or comment out the budget constraint entirely.